ML Vision **S**tudio

Developed by Darryl Arun David,

Guided by Prof. Ankur Miglani and Prof. Pavan Kumar Kankar

Overview

1. Purpose
2. How it works
3. What is involved
4. Critiques
5. Conclusion

Purpose

- Provide an intuitive graphical user interface for model training.
- Targets image-based classification using popular CNN models.
- Minimize need for coding.
- Simplifying model creation process for our lab.
- Currently supported models: MobileNet, ResNet, and Inception

Domains

- Human Computer Interaction (HCI)
- Software development, cross-platform apps
- Convolution Neural Networks for image classifications
- Servers and Linux systems

- Based on: Deep CNN-based damage classification of milled rice grains using a high-magnification image dataset
- Computer and electronics in agriculture

Purpose

Typical lifecycle of a model

Creation Tuning Usage (Inference)

Purpose

Our focus

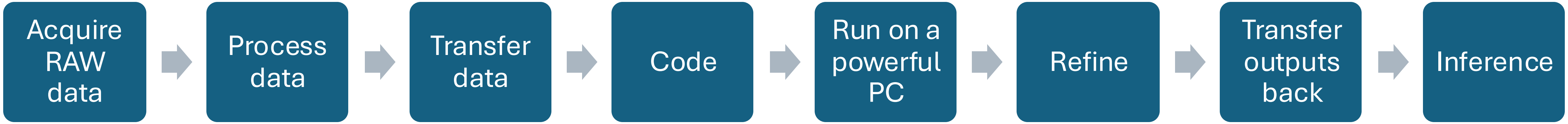
Creation

Tuning

Usage (Inference)

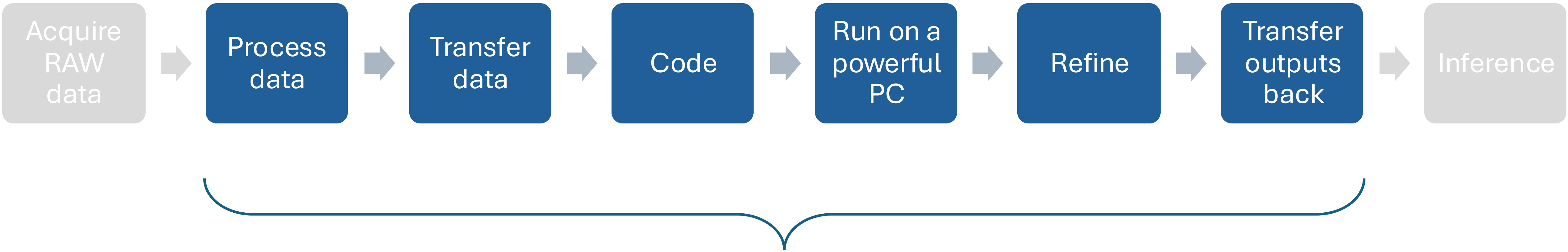
Purpose

Typical process of model creation



Purpose

Our focus

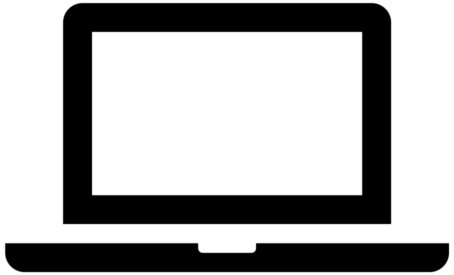


Simplify the process, and make it easy for non-coders,
while not compromising on power

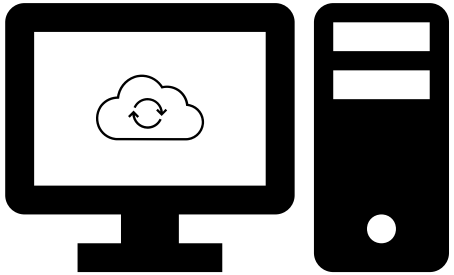
How it works

Two parts:

- One part runs as a fully graphical desktop app on <u>any computer</u>
- The other part is a suite of programs that run on a <u>powerful computer</u>



Any computer



Powerful computer for training

Both can run on the same computer too

How it works

First part:

- A user-friendly app with a graphical user interface.

Any computer

What it does:

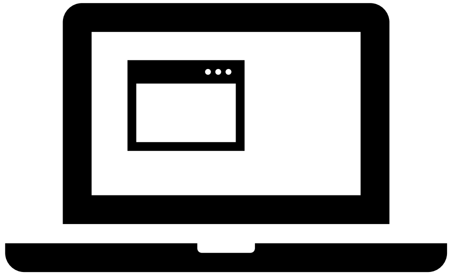
- Built-in image resizing and cropping, with caching
- Internal storage

- 4 easy steps to create a training set:

How it works

Part 1 of MLVS

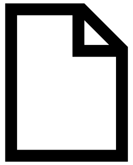
Any computer



Part 2 of MLVS

Powerful computer for training

Automatically creates a zip file with a folder structure for the training set, with the chosen configs in a Json file



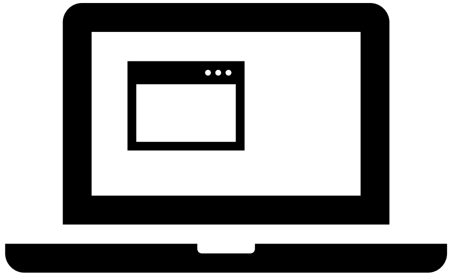
Sends for training

Takes the training set, executes training, and stores the outputs and model metrics

What is involved

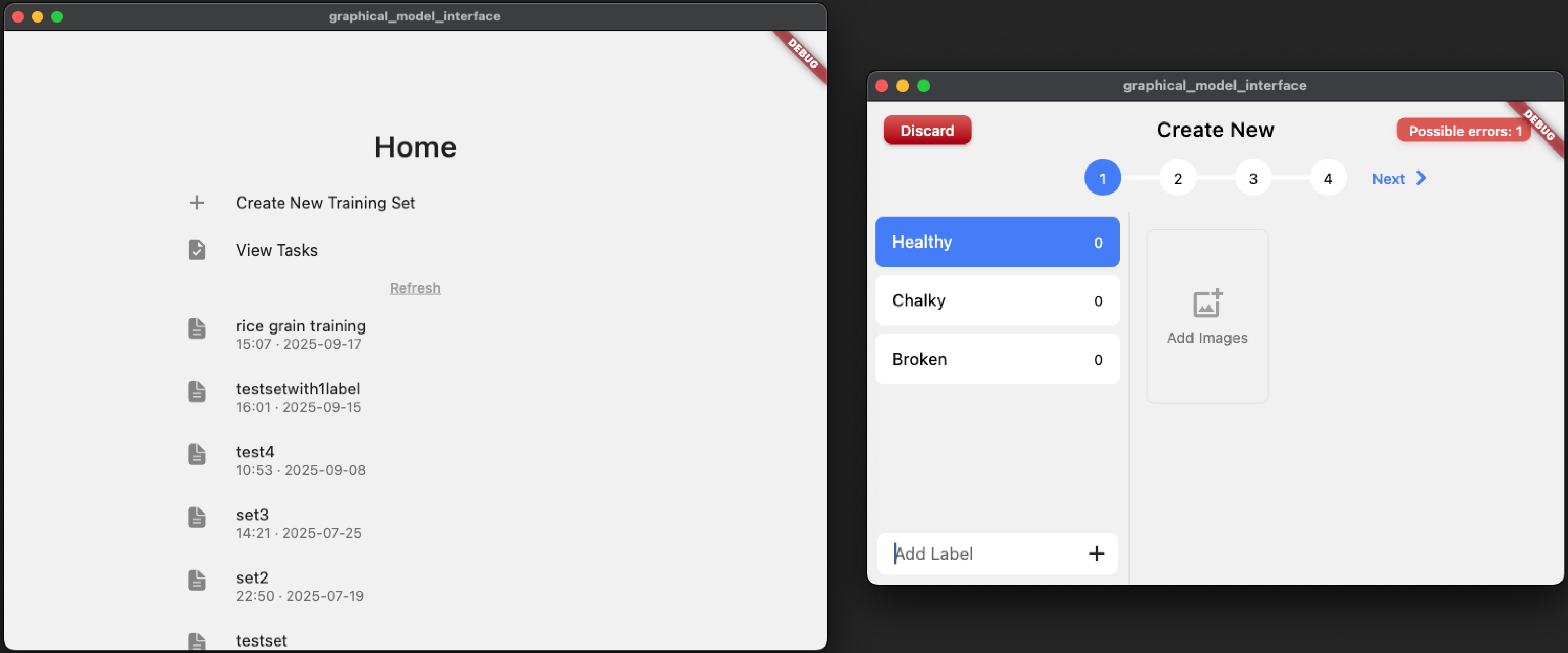
First part:

- A desktop program, that works on Windows, macOS, and Linux
- Works on Android, iOS, and Web, with limited functionality
- Fully graphical
- Made using Flutter



- Uses HTTP to send files to your server

- Json-based configs.
- GUI for parameters can be changed easily


graphical_model_interface
DEBUG
Home
Create New Training Set
View Tasks
Refresh
rice grain training
15:07 · 2025-09-17
testsetwith1label
16:01 · 2025-09-15
test4
10:53 · 2025-09-08
set3
14:21 · 2025-07-25
set2
22:50 · 2025-07-19
testset
graphical_model_interface
Discard
Create New
Possible errors: 1
DEBUG
1
2
3
4
Next
Healthy 0
Chalky 0
Broken 0
Add Images
Add Label

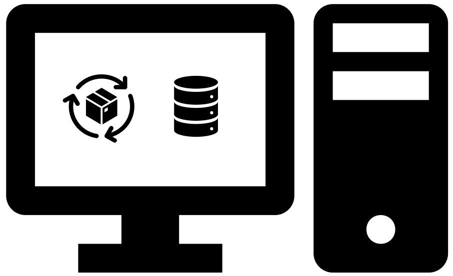
What is involved

Second part:

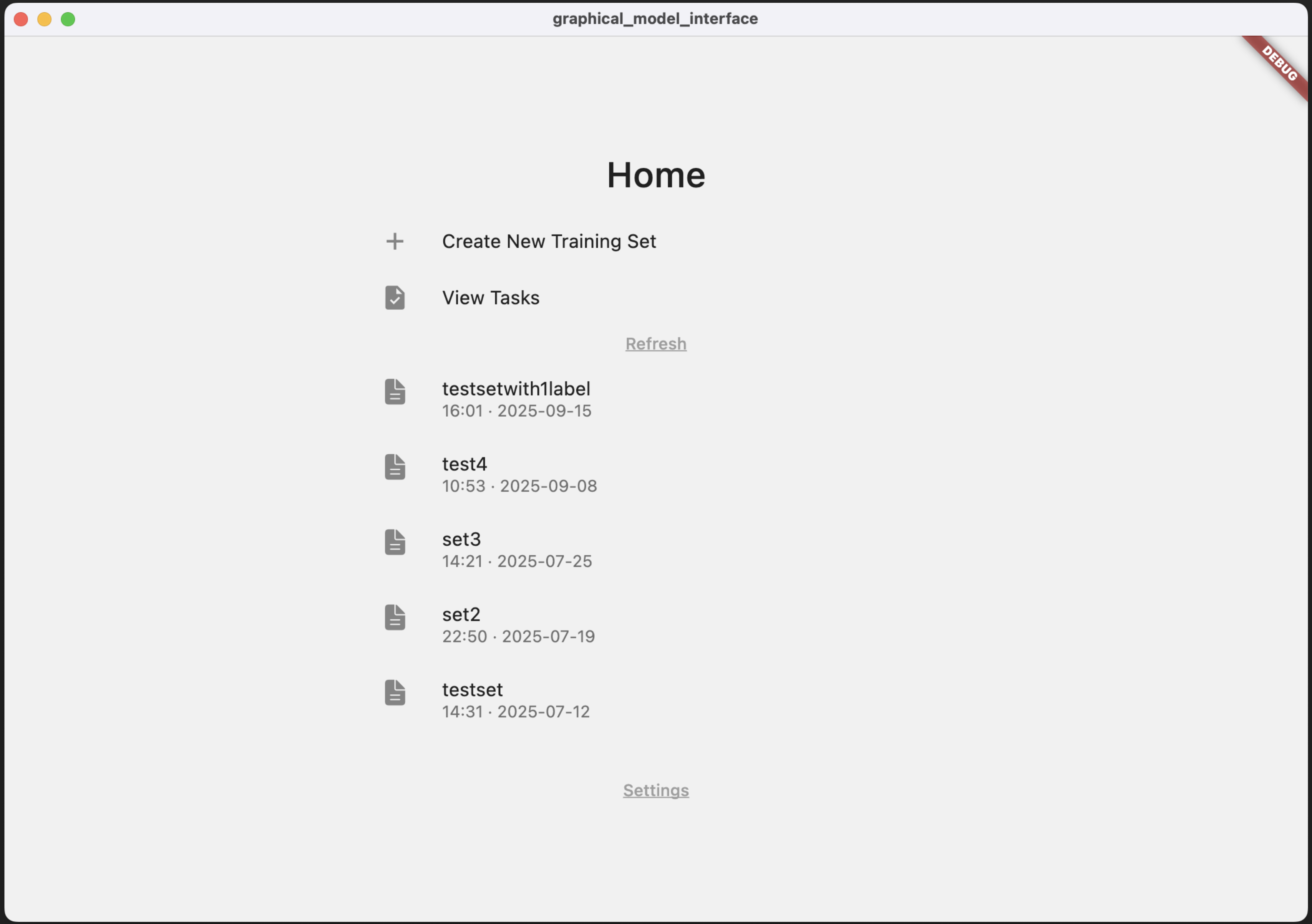
- A collection of softwares, meant to run on a powerful computer
- Fully Dockerized, can run on any computer with Docker
- No graphical interface, designed to run headless

- Server framework used: FastAPI
- Task management used by Redis,
- Tasks executed using Celery workers

- Restricted to campus network



How it works


graphical_model_interface
DEBUG
Home
Create New Training Set
View Tasks
Refresh
testsetwith1label
16:01 · 2025-09-15
test4
10:53 · 2025-09-08
set3
14:21 · 2025-07-25
set2
22:50 · 2025-07-19
testset
14:31 · 2025-07-12
Settings

graphical_model_interface

Discard

Create New

Possible errors: 1

DEBUG

1 2 3 4 Next

healthy	147
chalky	126
broken	91
discolored	91

Add Label +

4d333333333]5d33...

4d333333333]5d33...

4d333333333]5d33...

4d333333333]5d33...

4d333333333]5d33...

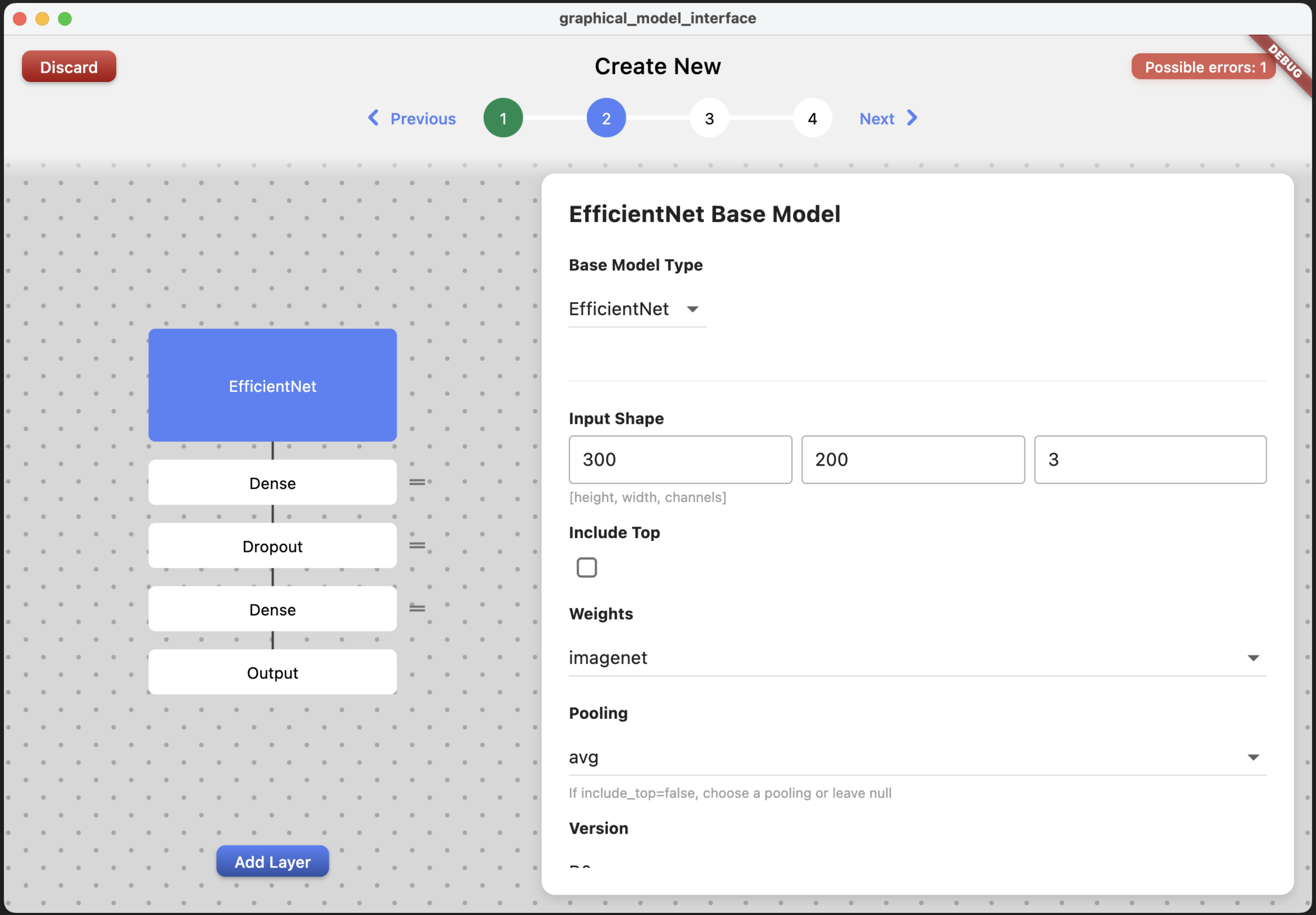
4d444444444]5d44...

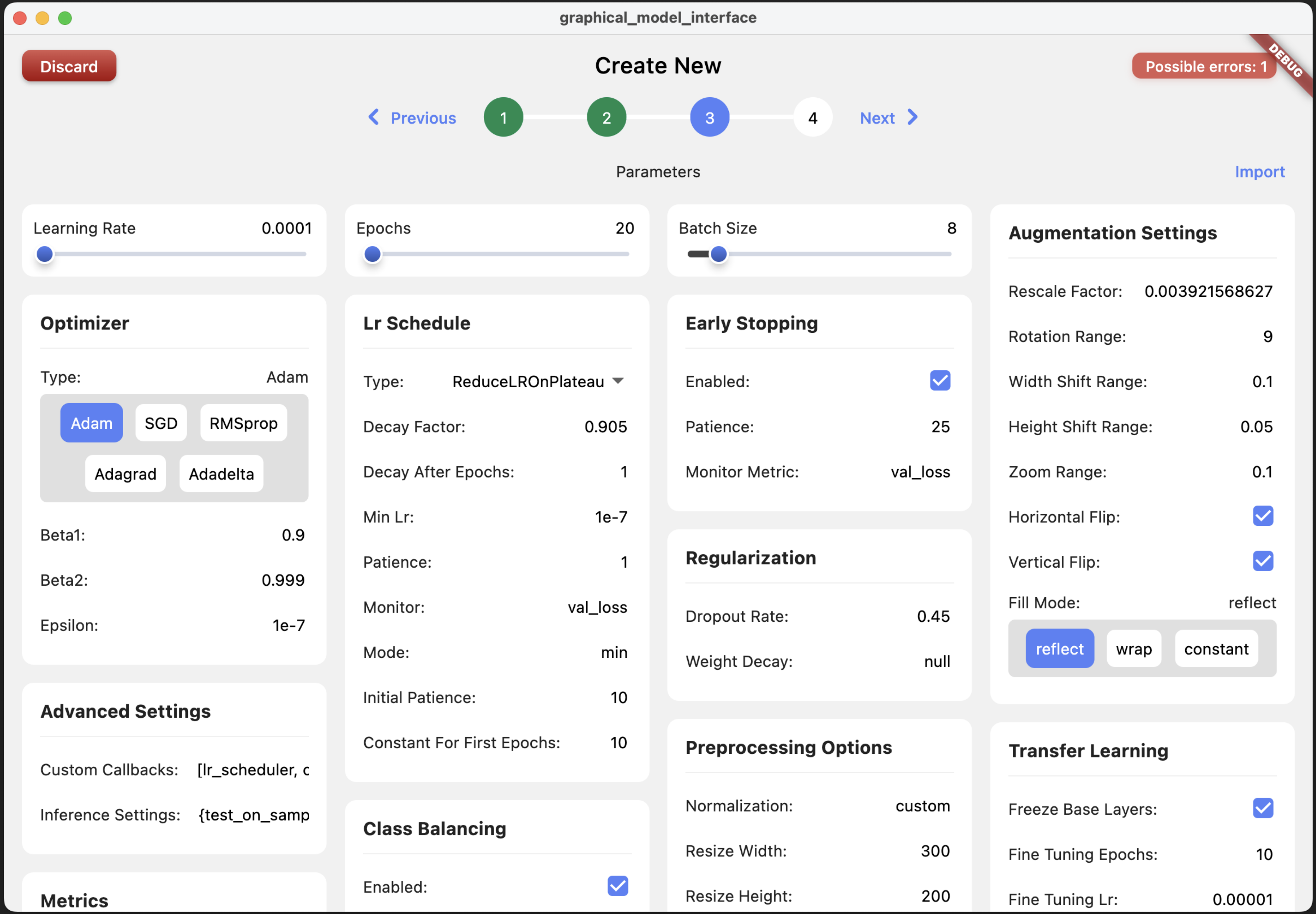
4d444444444]5d44...

4d444444444]5d44...

4ds5ds6ds1.jpg

4ds5ds6ds2.jpg


graphical_model_interface
Discard
Create New
Possible errors: 1
DEBUG
Previous
1
2
3
4
Next
EfficientNet
Dense
Dropout
Dense
Output
Add Layer
EfficientNet Base Model
Base Model Type
EfficientNet
Input Shape
300
200
3
[height, width, channels]
Include Top
Weights
imagenet
Pooling
avg
If include_top=false, choose a pooling or leave null
Version


graphical_model_interface
Discard
Create New
Possible errors: 1
DEBUG
Previous
1
2
3
4
Next
Parameters
Import
Learning Rate 0.0001
Epochs 20
Batch Size 8
Augmentation Settings
Rescale Factor: 0.003921568627
Rotation Range: 9
Width Shift Range: 0.1
Height Shift Range: 0.05
Zoom Range: 0.1
Horizontal Flip:
Vertical Flip:
Fill Mode: reflect
reflect
wrap
constant
Optimizer
Type: Adam
Adam
SGD
RMSprop
Adagrad
Adadelta
Beta1: 0.9
Beta2: 0.999
Epsilon: 1e-7
Lr Schedule
Type: ReduceLROnPlateau
Decay Factor: 0.905
Decay After Epochs: 1
Min Lr: 1e-7
Patience: 1
Monitor: val_loss
Mode: min
Initial Patience: 10
Constant For First Epochs: 10
Early Stopping
Enabled:
Patience: 25
Monitor Metric: val_loss
Regularization
Dropout Rate: 0.45
Weight Decay: null
Advanced Settings
Custom Callbacks: [lr_scheduler, c
Inference Settings: {test_on_samp
Class Balancing
Enabled:
Preprocessing Options
Normalization: custom
Resize Width: 300
Resize Height: 200
Transfer Learning
Freeze Base Layers:
Fine Tuning Epochs: 10
Fine Tuning Lr: 0.00001
Metrics


graphical_model_interface
Discard
Create New
Possible errors: 1
DEBUG
Previous
1
2
3
4
Next
Split
65.12%
34.88%
Train
Testing + Cross Validation
66.7%
33.3%
Cross Validation
Testing
Shuffle Data

graphical_model_interface

Exit

DEBUG

Export complete

4 labels, 455 images

Processing: 100% complete

Save As

Model configuration saved to library

graphical_model_interface

rice grain training

DEBUG

Created: 2025-09-17 15:07

File Size	54.5 MB
Labels	4
Total Images	455

Labels:

- healthy
- chalky
- broken
- discolored

Save Training Set To ...

Send to Server

Delete

Note: Deleting this configuration will also delete the associated dataset file.

Configuration


```
{
  "modelConfig": {
    "config_version": "1.0",
    "config_name": "rice grain training",
    "model_type": "EfficientNet",
    "params": {
      "learning_rate": 0.0001,
      "epochs": 20,
      "batch_size": 8,
      "augmentation_settings": {
        "rescale_factor": 0.003921568627,
        "rotation_range": 9,
        "width_shift_range": 0.1,
        "height_shift_range": 0.05,
        "zoom_range": 0.1,
        "horizontal_flip": true,
        "vertical_flip": true,
        "fill_mode": "reflect"
      },
      "optimizer": {
        "type": "Adam",
        "beta1": 0.9,
        "beta2": 0.999,
        "epsilon": 1e-7
      },
      "lr_schedule": {
        "type": "ReduceLROnPlateau",
        "decay_factor": 0.905,
        "decay_after_epochs": 1,
        "min_lr": 1e-7,
        "patience": 1,
        "monitor": "val_loss",
        "mode": "min",
        "initial_patience": 10,
        "constant_for_first_epochs": 10
      },
      "early_stopping": {
        "enabled": true,
        "patience": 25,
        "monitor_metric": "val_loss"
      },
      "regularization": {
        "dropout_rate": 0.45,
        "weight_decay": "null"
      },
```

graphical_model_interface

DEBUG

Tasks

Refresh

research_paper_config 05:34 · 2025-09-08 · EfficientNet	COMPLETED
research_paper_config 09:03 · 2025-07-25 · EfficientNet	COMPLETED
research_paper_config 17:35 · 2025-07-23 · EfficientNet	COMPLETED
research_paper_config 17:33 · 2025-07-23 · EfficientNet	FAILED
research_paper_config 13:55 · 2025-07-23 · EfficientNet	COMPLETED
research_paper_config 10:05 · 2025-07-23 · EfficientNet	COMPLETED
research_paper_config 11:42 · 2025-07-20 · EfficientNet	COMPLETED
research_paper_config 10:27 · 2025-07-20 · EfficientNet	COMPLETED

graphical_model_interface

research_paper_config

COMPLETED

DEBUG

Job ID: a1a843ba-c329-43ff-91c4-d87528f9b178
Created At: 2025-09-08T05:34:37.451769
Model Type: EfficientNet

Download all

trained_model.h5
50.64 MB

best_model.h5
50.64 MB

training_history.json
0.01 MB

model_architecture.json
0.01 MB

training_curves.png
0.81 MB

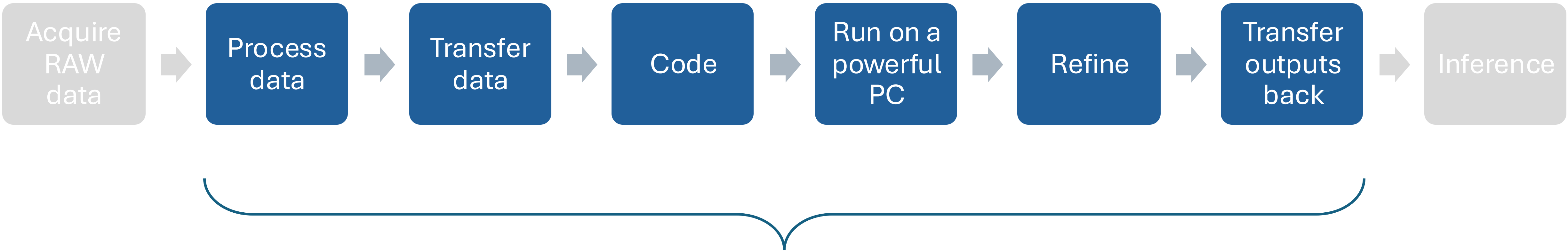
confusion_matrix.csv
0.0 MB

classification_report.json
0.0 MB

training_log.txt
0.53 MB

Achievement

Results



Simplify the process, and make it easy for non-coders,
while not compromising on power

Achievement

Results

Works without internet! Works within campus Wi-Fi (for now)

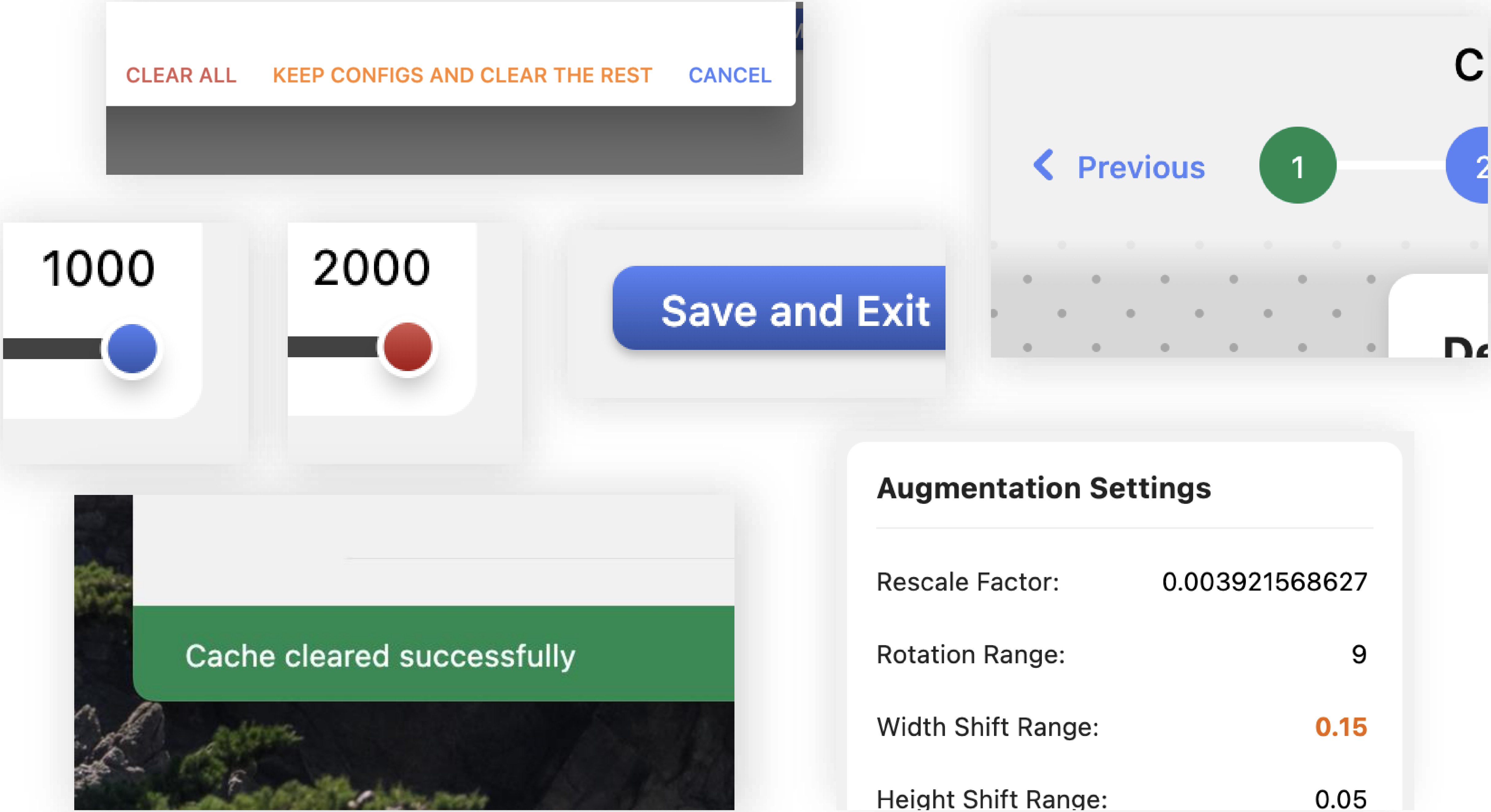
To be implemented …

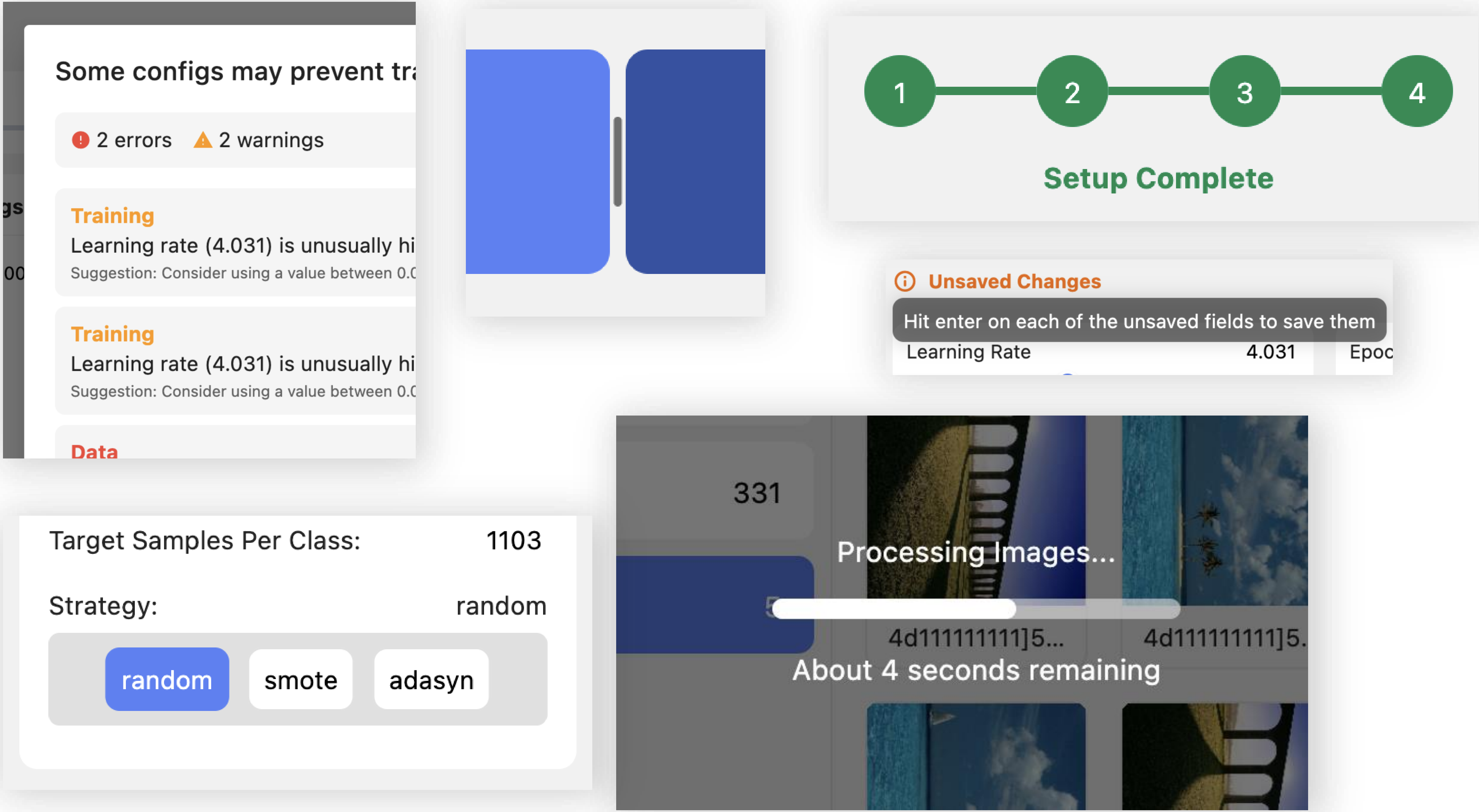
1. Send button
2. View live training stats in the app
3. Delete completed tasks

Critiques

1. No login system yet
2. Only designed for image-based models.
3. Software is modular, but to extend functionality, person needs to code.

Human Computer Interaction (HCI) and User Interface Design


CLEAR ALL
KEEP CONFIGS AND CLEAR THE REST
CANCEL
Previous
1
1000
2000
Save and Exit
Cache cleared successfully
Augmentation Settings
Rescale Factor: 0.003921568627
Rotation Range: 9
Width Shift Range: 0.15
Height Shift Range: 0.05


Some configs may prevent tra
2 errors
2 warnings
Training
Learning rate (4.031) is unusually hi
Suggestion: Consider using a value between 0.0
Training
Learning rate (4.031) is unusually hi
Suggestion: Consider using a value between 0.0
Data
1
2
3
4
Setup Complete
Unsaved Changes
Hit enter on each of the unsaved fields to save them
Learning Rate
4.031
Epoc
Target Samples Per Class:
1103
Strategy:
random
random
smote
adasyn
331
Processing Images...
4d111111111]5...
4d111111111]5.
About 4 seconds remaining

ML Vision Studio

Developed by Darryl Arun David,

Guided by Prof. Ankur Miglani and Prof. Pavan Kumar Kankar